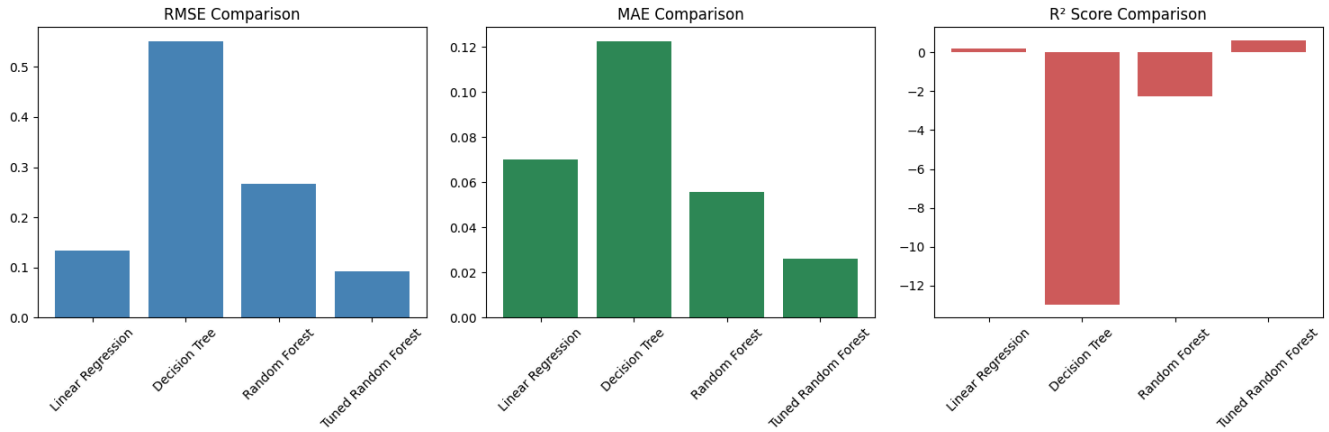
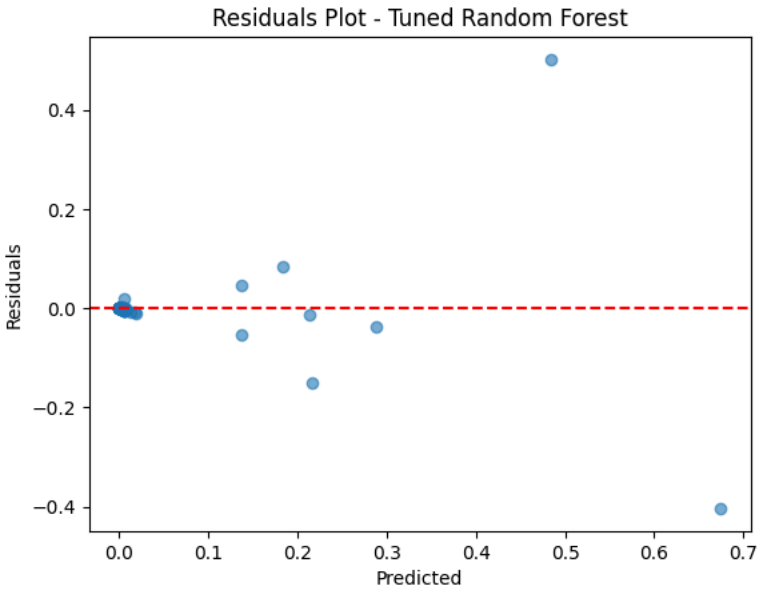
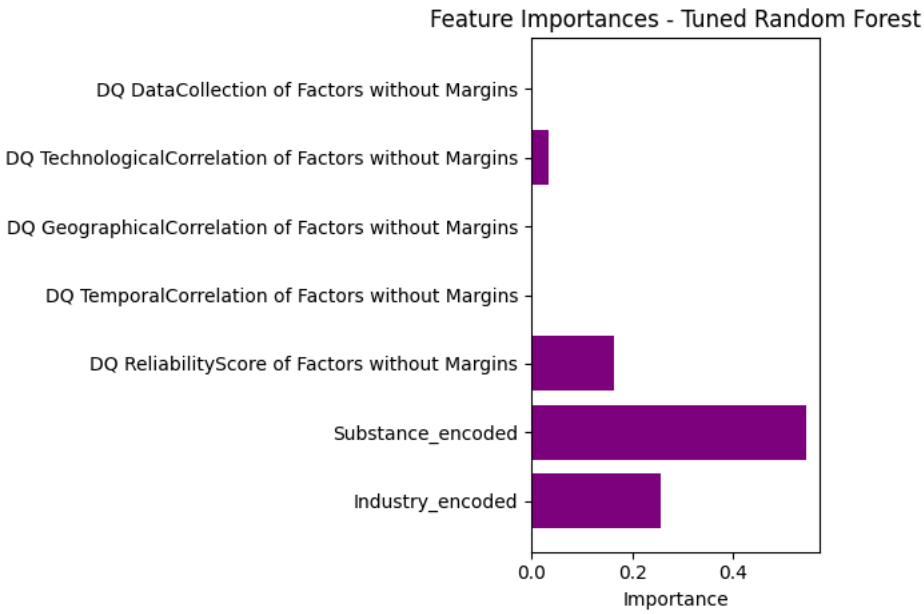
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.preprocessing import LabelEncoder
import joblib

# 2. Load the Excel file
# After upload, your file should be in /content/ directory,
# change filename below to your uploaded file's exact name

file_path = '/content/SupplyChainEmissionFactorsforUSIndustriesCommodities(2015_Summary_Industry).xls'

# Read Excel file
df = pd.read_excel(file_path)

# 3. Data preprocessing
df.columns = df.columns.str.strip()
df = df.dropna(subset=['Supply Chain Emission Factors with Margins'])
df['Supply Chain Emission Factors with Margins'] = pd.to_numeric(
    df['Supply Chain Emission Factors with Margins'], errors='coerce')
df = df.dropna(subset=['Supply Chain Emission Factors with Margins'])

le_industry = LabelEncoder()
le_substance = LabelEncoder()

df['Industry_encoded'] = le_industry.fit_transform(df['Industry Name'])
df['Substance_encoded'] = le_substance.fit_transform(df['Substance'])

features = [
    'Industry_encoded', 'Substance_encoded',
    'DQ ReliabilityScore of Factors without Margins',
    'DQ TemporalCorrelation of Factors without Margins',
    'DQ GeographicalCorrelation of Factors without Margins',
    'DQ TechnologicalCorrelation of Factors without Margins',
    'DQ DataCollection of Factors without Margins'
]

X = df[features]
y = df['Supply Chain Emission Factors with Margins']

# 4. Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 5. Train models
models = {
    'Linear Regression': LinearRegression(),
    'Decision Tree': DecisionTreeRegressor(random_state=42),
    'Random Forest': RandomForestRegressor(random_state=42)
}

results = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    results[name] = {
        'RMSE': np.sqrt(mean_squared_error(y_test, preds)),
        'MAE': mean_absolute_error(y_test, preds),
        'R2': r2_score(y_test, preds)
    }
    print(f"{name}: RMSE={results[name]['RMSE']:.4f}, MAE={results[name]['MAE']:.4f}, R2={results[name]['R2']:.4f}")

# 6. Tune Random Forest
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

grid = GridSearchCV(RandomForestRegressor(random_state=42), param_grid, cv=5, scoring='neg_mean_squared_error', n_jobs=-1)
grid.fit(X_train, y_train)

best_rf = grid.best_estimator_
preds_best = best_rf.predict(X_test)
```

```
results['Tuned Random Forest'] = {
    'RMSE': np.sqrt(mean_squared_error(y_test, preds_best)),
    'MAE': mean_absolute_error(y_test, preds_best),
    'R2': r2_score(y_test, preds_best)
}

print(f"Tuned Random Forest: RMSE={results['Tuned Random Forest']['RMSE']:.4f}, MAE={results['Tuned Random Forest']['MAE']:.4f}, R2={resu

# 7. Visualize results
names = list(results.keys())
rmse_vals = [results[name]['RMSE'] for name in names]
mae_vals = [results[name]['MAE'] for name in names]
r2_vals = [results[name]['R2'] for name in names]

plt.figure(figsize=(15,5))
plt.subplot(1,3,1)
plt.bar(names, rmse_vals, color='steelblue')
plt.title('RMSE Comparison')
plt.xticks(rotation=45)

plt.subplot(1,3,2)
plt.bar(names, mae_vals, color='seagreen')
plt.title('MAE Comparison')
plt.xticks(rotation=45)

plt.subplot(1,3,3)
plt.bar(names, r2_vals, color='indianred')
plt.title('R² Score Comparison')
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

# 8. Residual plot for tuned RF
residuals = y_test - preds_best
plt.scatter(preds_best, residuals, alpha=0.6)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Predicted')
plt.ylabel('Residuals')
plt.title('Residuals Plot - Tuned Random Forest')
plt.show()

# 9. Feature importance
importances = best_rf.feature_importances_
plt.barh(features, importances, color='purple')
plt.xlabel('Importance')
plt.title('Feature Importances - Tuned Random Forest')
plt.tight_layout()
plt.show()

# 10. Save the best model
import joblib
joblib.dump(best_rf, 'best_random_forest_model.pkl')
print("Best Random Forest model saved to 'best_random_forest_model.pkl'")
```

```
Linear Regression: RMSE=0.1341, MAE=0.0698, R2=0.1763
Decision Tree: RMSE=0.5519, MAE=0.1225, R2=-12.9520
Random Forest: RMSE=0.2675, MAE=0.0557, R2=-2.2771
Tuned Random Forest: RMSE=0.0923, MAE=0.0262, R2=0.6101
```








```
Best Random Forest model saved to 'best_random_forest_model.pkl'
```